

Exercicis Resolts. PRO1

Professorat de PRO1. Versió 1.1

December 14, 2021

Contents

1	Versions	3
2	Introducció	3
3	Notació	3
4	Algoritmes Iteratius	4
4.1	Altura d'una pila	4
4.1.1	Solució abreujada	6
4.2	Cerca en una Cua	8
4.2.1	Solució abreujada	11
4.3	Sumar k a tots els Element d'una Llista	13
4.3.1	Solució abreujada	15
4.4	Llista Acumulada	17
4.5	Intercala Piles (Punters)	20
5	Algoritmes Recursius	24
5.1	Fulles Esqueres	24
5.2	Reordena Pila	27
5.3	Arbre Creixent	29
5.4	Llista Ordenada per Paritat	32
5.5	Pila Ordenada	35
5.6	Inserció en Arbre de Cerca (Punters)	38

1 Versions

1. Versió 1.0: 3 problemes + 3 problemes examen parcial Q2 2020-21.
2. Versió 1.1: afegits 3 problemes examen parcial Q1 2021-22.
3. Versió 1.2: afegit Intercala Piles (Punters).
4. Versió 1.3. afegit Inserció en Arbre de Cerca (Punters).

2 Introducció

En aquest document hi ha exercicis de verificació d'algoritmes de l'assignatura PR01 de l'Enginyeria en Informàtica, tant de programes iteratius com recursius. N'hi ha que són exercicis d'examen i d'altres no.

Les explicacions que es donen en molts casos són detallades, i en certa manera, es fan servir formalismes per a assegurar-nos de la correctesa del que volem demostrar. Quan les explicacions són molt llargues, oferim una solució abreujada, que s'acosta més a allò mínimament exigible en un examen per a considerar la resposta com a satisfactòria.

3 Notació

Fem servir la següent notació:

- $\wedge$ és l'operador *i lògic* (en anglès: *and*).
- $\vee$ és l'operador *o lògic* (en anglès: *or*).
- $\forall$ és el quantificador universal, que és una conjunció de totes les clàusules que es poden formar amb el rang definit. Per exemple:

$$\bigwedge_{0 \leq i < n} P(i) = P(0) \wedge P(1) \wedge P(2) \wedge \dots \wedge P(n-1)$$

4 Algoritmes Iteratius

4.1 Altura d'una pila

Cal verificar una funció que, donada una pila, en calculi l'altura. Aquest problema rebrà una pila per còpia, i tornarà un enter. Per a especificar aquest problema farem servir la notació $|S|$, tal que, donada una estructura, avalua al nombre d'elements que aquesta estructura conté (ve a ser l'equivalent del `.size()` en `C++`). Com ja sabreu, aquesta notació és una sobrecàrrega de la notació que es fa servir per a denotar el nombre d'elements d'un conjunt. En aquest cas, l'altura d'una pila equival al nombre d'elements.

Fixeu-vos que això només passa en piles, llistes, cues i estructures lineals, però no en altres estructures com, per exemple, arbres binaris, en què l'alçada no ha de correspondre necessàriament amb el nombre d'elements de l'arbre, o l'amplada, per exemple.

El programa que proposem és el següent:

```
1 int alturaPila (stack <int> p)
2 // Pre : p = P
3 {
4     int r = 0 ;
5
6     while (not p.empty())
7     {
8         ++r;
9         p.pop();
10    }
11
12    return r;
13
14 // Post: r = |P|
15 }
```

Com podeu veure, és un esquema de recorregut, que simplement va desempilant elements de la pila i va incrementant el comptador `r`. Ara cal demostrar que aquest programa calcula relament la postcondició. Per a fer tal cosa, caldrà primer proposar un invariant. En aquest cas, l'invariant ha d'explicar què passa amb les variables que hi ha a dins del bucle, i, sobretot, les que intervenen en el càlcul del resultat. En aquest cas, veureu que les variables que ens ajuden a calcular el resultat són `r` i, col·lateralment, la pila. Per tant, ens podem centrar en explicar a dins de l'invariant **com calculem el resultat**. La intuïció és bastant clara: `r` tindrà el nombre d'elements que haurem desempila de la pila **original** `P`. Per tant, podem proposar aquest invariant:

$$r = |P| - |p|$$

Ara ja podem procedir a verificar que aquest programa calcula la postcondició. Primer de tot cal verificar que abans de la primera iteració es compleix l'invariant. Ara bé, fixem-nos que just abans de la línia 7 (**while**) tenim que es compleix la precondició (perquè no hem modificat la variable `p`) i també tenim que `r = 0`. Per tant:

$$\begin{array}{lll}
r = |P| - |p| \equiv & 0 = |P| - |p| & (r = 0) \\
\equiv & 0 = |P| - |P| & (PRE : p = P) \\
\equiv & 0 = 0 & \text{cert}
\end{array}$$

Ara toca verificar que l'invariant es manté invariant en cadascuna de les iteracions. Això es fa per inducció: assumim (com a hipòtesi d'inducció) que l'invariant és cert **abans** de començar una iteració, i verifiquem que després de la iteració ho segueix sent.

Per a poder-ho verificar, caldrà que recordem que si tenim una pila no buida que té mida n , si li desempilem un element tindrà mida $n - 1$.

$$\begin{array}{ll}
r = |P| - |p| \equiv & \text{hipòtesi d'inducció} \\
\equiv & r + 1 = |P| - |p| \quad (+ + r) \\
\equiv & r + 1 = |P| - (|p| - 1) \quad (p.pop()) \\
\equiv & r + 1 = |P| - |p| + 1 \quad \text{aritmètica} \\
\equiv & r = |P| - |p| \quad \text{Invariant}
\end{array}$$

Com veieu, hem agafat l'invariant **abans** de la iteració, li hem aplicat els canvis del codi a dins de la iteració, i hem obtingut l'invariant una altra vegada.

Cal tenir molt en compte que cal verificar que totes les operacions es poden executar sense errors. En aquest cas, l'operació que potencialment podria portar problemes és `p.pop()`, però com que només s'executa si la condició del bucle (not `p.empty()`) es compleix, llavors no tenim cap problema.

Ara cal demostrar que un cop acabem d'iterar, arribem a la postcondició. En aquest cas, com que no tenim cap instrucció entre el final del **while** i el **return**, només cal verificar que l'invariant i la negació de la condició del bucle impliquen la postcondició. Cal que recordem que la mida/altura d'una pila buida és zero:

$$\begin{array}{lll}
r = |P| - |p| \equiv & r = |P| - 0 & (p.empty()) \\
\equiv & r = |P| & \text{Postcondició}
\end{array}$$

Finalment, cal definir una funció fita, que decreixi estrictament a cada volta de la iteració. En aquest cas, podem agafar $|p|$. Primer veiem que és una funció sobre les naturals, ja que la mida d'una pila és sempre un natural, en segon lloc, cal demostrar que decreix estrictament a cada iteració. Això ho podem demostrar pel fet que a cada volta desempilem un element de la pila (i a més, sabent que $|p| = p.pop() + 1$ quan la pila no és buida, cosa que, per altra banda, podem verificar perquè la condició del bucle ens diu que not `p.empty()`).

4.1.1 Solució abreujada

1. PRE + inicialitzacions $\Rightarrow$ INV. La precondició ens diu que $\mathbf{p} = P$, i se segueix complint abans de la iteració perquè no hem modificat $\mathbf{p}$. Com que inicialitzem $\mathbf{r} = 0$, substituint a l'invariant $r = |P| - |p|$, tenim:

$$0 = |P| - |p|$$

cosa que és certa.

2. Condició del bucle + INV $\Rightarrow$ INV. Assumim (com a hipòtesi d'inducció) que l'invariant és cert **abans** de començar una iteració.

L'invariant que tindrem **després** de la iteració és el mateix, però amb les variables modificades segons el que haurem fet al cos del bucle. Això vol dir que ens quedarà aquest invariant:

$$r + 1 = |P| - |p.\text{pop}()|$$

per les instruccions de les línies 8 i 9. Si operem aquest invariant, veurem que podem transformar-lo en

$$r + 1 = |P| - |p| + 1$$

perquè sabem que si tenim una pila no buida que té mida n , si li desempilem un element tindrà mida $n - 1$. Per tant, ara ens queda

$$r = |P| - |p|$$

que és, precisament, l'invariant, és a dir, la hipòtesi d'inducció, que assumim que és certa.

També cal verificar que totes les operacions del cos del bucle es poden executar sense errors. En aquest cas, l'operació que potencialment podria portar problemes és $\mathbf{p}.\text{pop}()$, però com que només s'executa si la condició del bucle (**not** $\mathbf{p}.\text{empty}()$) es compleix, llavors no tenim cap problema.

3. Negació de la condició del bucle + INV $\Rightarrow$ POST. Com que es compleix $\mathbf{p}.\text{empty}()$ (la negació de la condició del bucle) llavors tenim que l'invariant ens queda

$$r = |P| - 0$$

perquè la mida d'una pila buida és 0. Això és, precisament, l'invariant.

4. Funció fita. En aquest cas, podem agafar $|p|$. Primer veiem que és una funció sobre les naturals, ja que la mida d'una pila és sempre un natural,

en segon lloc, cal demostrar que decreix estrictament a cada iteració. Això ho podem demostrar pel fet que a cada volta desempilem un element de la pila (i a més, sabent que $|p| = |p.pop()| + 1$ quan la pila no és buida, cosa que, per altra banda, podem verificar perquè la condició del bucle ens diu que `not p.empty()`).

4.2 Cerca en una Cua

El següent exemple ha de trobar un element en una cua. Això és, òbviament, un esquema de cerca, i ho podem implementar de la següent manera:

```
1 bool cercaCua(queue<int> c, int x)
2 // Pre: c = C
3 {
4     bool trobat = false;
5
6     while(not c.empty() and not trobat)
7     {
8         trobat = c.front() == x;
9         c.pop();
10    }
11
12    return trobat;
13 }
14 // Post: trobat = x ∈ C
```

Aquí tornem a fer un abús de notació: $x \in C$, que vol dir que la x és un element de la cua C . Com sempre, aquest abús de notació ho fem per tal que els càlculs quedin més clars. A més, cal que tingueu en compte que la postcondició la posem en funció del valor que tenia la cua al principi de la crida, ja que la modifiquem durant l'execució de la funció.

Per a verificar el programa, caldrà proposar un invariant. La tria natural de l'invariant consistiria en **explicar** què li passa a la variable **trobat** (i, col·lateralment, com en l'exemple de l'altura de la pila, amb la cua), que és el resultat que volem calcular. Una tria *informal* de l'invariant seria una cosa com *trobat és cert si i només si es troba en la part ja tractada de la cua c*. Això ja seria un invariant acceptable, però també podem proposar-ne uns altres d'alternatius. En aquest cas, podem tornar a abusar la notació i dir que l'invariant és

$$trobat = x \in C \setminus c$$

on $C \setminus c$ avalua a la cua C sense els elements que encara queden a la cua c , és a dir, a la part de la cua C que *ja hem processat*¹. Cal tenir en compte que C roman constant, mentre que c es va modificant (fent-se més petita) a cada volta de la iteració.

Ara ja podem verificar el programa. Primer de tot, cal verificar que **abans** de començar el bucle, l'invariant és cert.

¹Perquè aquesta operació estigui definida, caldria dir que C consisteix en la pila c més altres elements, cosa que donem per assegurat en aquest context

$$\begin{aligned}
trobat = x \in C \setminus c &\equiv false = x \in C \setminus c && (trobat = false;) \\
&\equiv false = x \in C \setminus C && (PRE : c = C) \\
&\equiv false = x \in \emptyset && (\text{definició de } C \setminus c) \\
&\equiv false = false && (x \text{ no pot ser en una estructura buida})
\end{aligned}$$

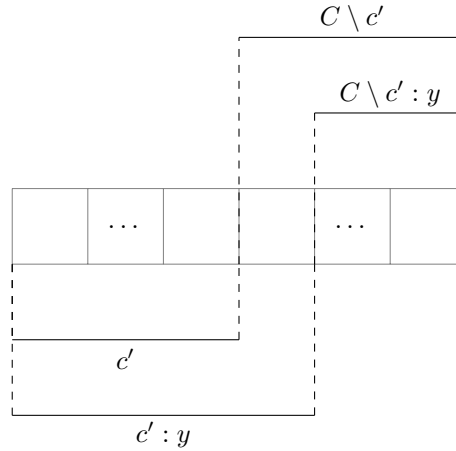
Ara cal verificar que l'invariant segueix sent cert després de qualsevol iteració. Recordeu que això vol dir que l'invariant i la condició del bucle, més l'acció de les instruccions dins del bucle impliquen l'invariant. Primer de tot, cal que tinguem en compte que la condició del bucle és certa, cosa que vol dir que:

$$x \notin C \setminus c$$

i que

$$\text{not } c.\text{empty}()$$

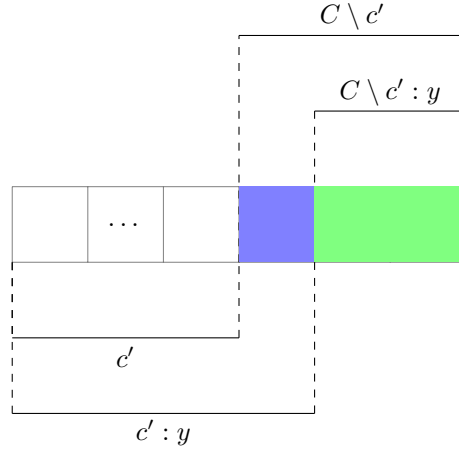
Podem aprofitar la darrera condició per a separar el cap de la cua c de la resta de la cua: $c = c' : y$. Ho podem veure en aquest dibuix:



Ara ja podem raonar sobre què passa a dins del codi:

$$\begin{aligned}
trobat = (x == y) &\equiv && (\text{on } y = c.\text{front}()) \\
&\equiv trobat = (x == y) \vee false && (P = P \vee fals) \\
&\equiv trobat = (x == y) \vee x \in C \setminus c' : y && (x \notin C \setminus c' : y) \\
&\equiv trobat = x \in C \setminus c && (c.\text{pop}())
\end{aligned}$$

Revisem els dos darrers passos d'aquesta verificació. Amb l'assignació $trobat = c.front() == x$; de la línia 8, el valor de **trobat** se sobreescriu amb l'expressió $c.front() == x$. Com que sabem que $P = P \vee fals$, podem substituir aquest **fals** amb qualsevol expressió que sigui falsa, per exemple, el que ja ens ha donat la hipòtesi d'inducció (és a dir, l'invariant **abans** d'entrar al bucle) i la condició del bucle: $x \notin C \setminus c' : y$. Perquè fem això? Perquè cal que tornem a tenir un invariant que raoni sobre **tota** la cua, i no només el final. Un cop hem fet això, i tenint en compte que escapem un element de la cua, tenim que $trobat = x == y$ or $x \in C \setminus c'$. Si ens fixem en el dibuix anterior, veurem que aquesta expressió vol dir que x es troba a la part verda ($C \setminus c' : y$) o bé a la part blava (y o, el que és igual, $c.front()$):



Podem dir que el fet de dir que es troba a la part blava o a la verda és una mica redundant, ja que la hipòtesi d'inducció i la condició de la iteració ens diu, precisament, que *no* es troba a la part verda, però cal tenir en compte que al final cal obtenir l'invariant $x \in C \setminus c'$, i que per tant, cal poder raonar sobre tota la part de la cua que hem tractat. De fet, la hipòtesi d'inducció ens està dient precisament que si entrem al bucle és perquè encara no hem trobat l'element que cerquem, cosa que és coherent amb el fet que estem aplicant un esquema de cerca.

Fixem-nos també que a dins del bucle hem calculat només la part que correspon a la part blava del dibuix, és a dir, hem avançat una sola posició de l'estructura que estem recorrent, en aquest cas, una cua.

Finalment, cal verificar que després de la darrera iteració, ja que no tenim cap altra instrucció abans del **return**, arribem a la postcondició. És a dir, que l'invariant i la negació de la condició del bucle implica a la postcondició. La negació de la condició (que, per tant, és una expressió que avalua a **true**) és:

$$trobat \vee c.empty()$$

tenint en compte una cosa important, i és que es tracta d'un or exclusiu, és a dir, o bé **trobat** avalua a cert i **c.empty()** avalua a fals, o bé **trobat** avalua

a fals i `c.empty()` avalua a cert. Per tant, quan acabem tenim només aquestes dues possibilitats:

1. **trobat** avalua a cert, i com que tenim que l'invariant és $trobat = x \in C \setminus c$, tenim que $x \in C \setminus c$, i com que es troba en una part de l'estructura, també es troba a dins de tota l'estructura: $trobat = x \in C$.
2. **trobat** avalua a fals, i llavors tenim que `c.empty()` és cert, amb la qual cosa, l'invariant $trobat = x \in C \setminus c$ esdevé $trobat = x \in C$, ja que `c` és buit.

Com veiem, en tots dos casos, arribem a la postcondició.

Finalment, cal definir una funció fita, que en aquest cas podria ser perfectament $|c|$, és a dir, el nombre d'elements de la cua `c`. Aquesta funció torna sempre un natural, i a més, decreix estrictament a cada iteració perquè li apliquem l'operació `c.pop()` tenint en compte que aquesta operació sempre està definida *dins del bucle* ja que ens trobem en la condició `not c.empty()`.

4.2.1 Solució abreujada

Proposta d'invariant: $trobat = x \in C \setminus c$. Demostrem:

1. Precondició + inicialitzacions $\Rightarrow$ INV. És a dir: $c = C \wedge trobat = false \Rightarrow trobat = x \in C \setminus c$. Com que $c = C$, tenim que $C \setminus c = \emptyset$, i per tant, $x \in \emptyset$ és fals. Per altra banda tenim que $trobat = false$. Això vol dir que l'invariant $trobat = x \in C \setminus c$ és cert, ja que és fals que x sigui en una estructura buida.
2. Condició del bucle + INV $\Rightarrow$ INV. La condició del bucle ens diu que $trobat = false$, que la cua no és buida, i l'invariant ens diu que trobat és cert si i només si x és a la part de la cua C que ja hem processat. Per tant, la conjunció de totes aquestes condicions ens diu que x **no es troba** a la part de la cua C que ja hem processat.

Per altra banda, el que cal verificar és que partint d'aquestes condicions, i després d'executar una volta del bucle, l'invariant segueix sent cert. L'invariant ens diu que x és cert si i només si $x \in C \setminus c$. Cal veure que amb la instrucció `c.pop()` el que fem és incrementar en una posició l'interval de cerca: $C \setminus c$. Per tant, i tenint en compte que fins ara no hem trobat x , això només vol dir que si x és en aquest interval, només podrà ser aquest nou element que avancem. I això ho comprovem en la instrucció `trobat = c.front() == x;`.

Cal veure també que totes les operacions que fem a dins del bucle, així com la condició del bucle, són avaluable sense error. La condició no té cap instrucció que no sigui total, mentre que el cos del bucle en té dues: `c.front()` i `c.pop()`, però tenint en compte que ens trobem sota la condició que `not c.empty()`, no tindrem cap error.

3. Negació de la condició + $INV \Rightarrow POST$. La negació de la condició ens diu que, o bé *trobat* és cert i llavors hem trobat l'element que cercàvem, amb la qual cosa, la postcondició $trobat = x \in C$ es compleix, o bé que **trobat** és fals, i llavors, com que som en un esquema de cerca, vol dir que **c.empty()** és cert. Com que l'invariant és $trobat = x \in C \setminus c$ i *c* és buit, llavors l'invariant queda com $trobat = x \in C$, que és justament la postcondició.
4. Funció fita. En aquest cas, triem la funció $|c|$. Veiem que a cada pas *dins del bucle* aquesta funció decreix estrictament, ja que $|c| = 1 + |c.pop()|$, i això és cert perquè podem avaluar sempre **c.pop()** per trobar-nos dins de la condició **not c.empty()**.

4.3 Sumar k a tots els Element d'una Llista

Es tracta de fer un programa que, donada una llista per referència (que en aquest cas serà un paràmetre d'entrada i sortida) i un enter k , modifiqui la llista de manera que a tots els elements de la llista els haurem sumant k . Això és clarament un esquema de recorregut (sobre una estructura lineal), en aquest cas, sobre una llista. El programa que proposem és el següent:

```

1  int sumaK (list <int>& l)
2  // Pre : l = L
3  {
4      list<int>::iterator it2;
5      it2 = l.begin();
6
7      while(it2 != l.end())
8      {
9          *it2 = *it2 + k;
10         ++it2;
11     }
12
13     return;
14
15 // Post :  $\bigvee_{l.begin() \leq it < l.end()} (*it = L[it] + k)$ 
16 }
```

Aquest és un model de programa prototípic de recorregut en una estructura lineal.² La idea que farem servir serà sempre la mateixa: l'invariant serà un debilitament de la postcondició, i bàsicament ens dirà que hem calculat el resultat fins a la part que portem recorreguda d'aquesta estructura. Per tant, una proposta d'invariant podria ser aquesta:

$$\left[\bigvee_{l.begin() \leq it < it2} (*it = L[it] + k) \right]$$

Com veureu, aquí tornem a fer un cert abús de notació amb l'expressió $L[it]$, que avalua al valor que té la llista L a la posició que li correspon l'iterador it . Fixem-nos que aquí tant L com l tenen la mateixa mida, perquè no modifiquem la variable l . Tenint en compte això i el fet que mantindrem els iteradors sempre dins del rang que els correspon, podem fer servir l'expressió $L[it]$.

Aquest invariant ens està dient justament que ha fet la part del càlcul que se suposa que volem tenir al final (a la post) però només fins a l'interval que ve marcat per l'iterador $it2$.³

Ara podem començar la verificació.

1. Precondició + inicialitzacions $\Rightarrow$ INV.

En aquest cas, cal verificar que es compleix l'invariant, tenint en compte que el primer que fem és l'assignació $it2 = l.begin()$; llavors l'invariant queda així:

²Per *estructura lineal* volem dir qualsevol estructura en què podem passar per tots els elements un després de l'altre, en un ordre determinat

³En el cas de llistes, l'interval el marcarà un iterador, en el cas d'un vector, un subíndex, etc.

$$\left[\bigvee_{l.begin() \leq it < l.begin()} (*it = L[it] + k) \right]$$

Tenint en compte que aquest interval és buit, llavors aquesta expressió equival al neutre de l'operador lògic **and** que és **cert**.

2. Condició del bucle + INV $\Rightarrow$ INV. En aquest cas, cal veure que si complim l'invariant abans d'entrar al bucle, també el complirem després de la iteració. En aquest cas, ens trobem en el cas en què **abans** del bucle, l'invariant (la hipòtesi d'inducció) ens dirà quina part ja portem calculada, i l'invariant que haurem de verificar que avalua a cert, és a dir, l'invariant **després** de la iteració, ens diu quina part hem de tenir calculada. Per tant, tota la verificació consistirà simplement a verificar que en el cos del bucle hem calculat la part de la llista que falta. En aquest cas, com que haurem avançat només una posició (línia 10), llavors caldrà veure que hem calcula el resultat sobre aquesta (petita) part (cosa que fem a la línia 9).

Agafem l'invariant que cal verificar i mirem si avalua a cert:

$$\begin{aligned} \left[\bigvee_{l.begin() \leq it < it2+1} (*it = L[it] + k) \right] &\equiv (+ + it2) \\ \left[\bigvee_{l.begin() \leq it < it2} (*it = L[it] + k) \right] \wedge *(it2) = L[it2] + k &\equiv \text{(separem interval)} \\ cert \wedge *(it2) = L[it2] + k &\equiv \text{(INV abans del bucle)} \\ cert \wedge cert &\equiv (*it2 = *it2 + k;) \end{aligned}$$

Fixem-nos en els passos anteriors: a la primera línia tenim l'invariant que volem verificar (el que va al final de la iteració). Veiem que com que hem fet (al cos de la iteració) un increment $(++it2)$, això implica que en aquest invariant hem de tenir que l'abast del quantificador universal $\forall$ ha de ser $l.begin() \leq it < it2 + 1$. A la segona línia, dividim aquest quantificador en dues parts: la que va de $l.begin() \leq it < it2$ i la des darrer element, que és $it2$ (recordeu que tenim $it < it2 + 1$, és a dir, que el darrer valor de l'iterador **it** és **it2**). Com veieu, això ho fem perquè la primera part és, precisament, l'invariant abans d'entrar a la iteració que, com a hipòtesi d'inducció, hem assumit que és cert. Per tant, a la tercera línia, veiem que simplement ens queda verificar que $*(it2) = L[it2] + k$, cosa que es verifica perquè és precisament el que la l'assignació de la línia 9 de la funció que hem escrit.

Finalment cal tenir en compte que les operacions dins del cos del bucle que no són totals $(++it2$ o l'accés a $*it2$) estan protegides per la condició del bucle.

3. Negació de la condició + $INV \Rightarrow POST$. Quan acabem tenim que es compleix la negació de la condició: $it2 == l.end()$. Si substituïm $it2$ per $l.end()$ a l'invariant tenim

$$\left[\bigvee_{l.begin() \leq it < l.end()} (*it = L[it] + k) \right]$$

que és, precisament, la postcondició de la funció.

4. Funció fita. Com a funció fita, en aquests casos, agafarem, de manera genèrica, la mida de la part de l'estructura que ens queda per tractar. En aquest cas, seria la funció tal que, donat l'iterador $it2$ ens torna el nombre d'elements que hi ha entre $it2$ i $l.end()$. Com que és una mica, llavors està definida en els naturals. A més, a cada pas de la iteració executem $++it2$, la qual cosa fa que el valor d'aquesta funció decreixi estrictament a cada pas de l'execució. Tinguem en compte que $++it2$ o l'accés a $*it2$ no són funcions totals (no estan definides quan $it2$ és igual a $l.end()$, però precisament això no pot passar perquè estem protegits per la condició del bucle.

4.3.1 Solució abreujada

En aquest cas, només abreugem el segon punt de la demostració, els altres són iguals al que hem fet anteriorment.

1. Precondició + inicialitzacions $\Rightarrow INV$.

En aquest cas, cal verificar que es compleix l'invariant, tenint en compte que el primer que fem és l'assignació $it2 = l.begin()$; llavors l'invariant queda així:

$$\left[\bigvee_{l.begin() \leq it < l.begin()} (*it = L[it] + k) \right]$$

Tenint en compte que aquest interval és buit, llavors aquesta expressió equival al neutre de l'operador lògic **and** que és **cert**.

2. Condició del bucle + $INV \Rightarrow INV$.

L'invariant que ens queda després de la iteració és l'invariant que tenim abans de la iteració, amb les variables modificades segons de l'invariant ha de ser $l.begin() \leq it < it2 + 1$, a causa de la instrucció $++it2$. Ens queda així:

$$\left[\bigvee_{l.begin() \leq it < it2} (*it = L[it] + k) \right] \wedge *(it2) = L[it2] + k$$

La part de l'esquerra és precisament la hipòtesi d'inducció (l'invariant abans de la iteració), i per tant, assumim que és cert. Per tant, només cal verificar $*(it2) = L[it2] + k$, que és el que fa la instrucció de la línia 9.

Finalment cal tenir en compte que les operacions dins del cos del bucle que no són totals ($++it2$ o l'accés a $*it2$) estan protegides per la condició del bucle.

3. Negació de la condició + INV $\Rightarrow$ POST. Quan acabem tenim que es compleix la negació de la condició: $it2 == l.end()$. Si substituïm $it2$ per $l.end()$ a l'invariant tenim

$$\left[\bigvee_{l.begin() \leq it < it2} (*it = L[it] + k) \right]$$

que és, precisament, la postcondició de la funció.

4. Funció fita. Com a funció fita, en aquests casos, agafarem, de manera genèrica, la mida de la part de l'estructura que ens queda per tractar. En aquest cas, seria la funció tal que, donat l'iterador $it2$ ens torna el nombre d'elements que hi ha entre $it2$ i $l.end()$. Com que és una mica, llavors està definida en els naturals. A més, a cada pas de la iteració executem $++it2$, la qual cosa fa que el valor d'aquesta funció decreixi estrictament a cada pas de l'execució. Tinguem en compte que $++it2$ o l'accés a $*it2$ no són funcions totals (no estan definides quan $it2$ és igual a $l.end()$), però precisament això no pot passar perquè estem protegits per la condició del bucle.

4.4 Llista Acumulada

Cal verificar una funció tal que, donada una llista d'enters més grans o iguals que zero per referència (serà, doncs, un paràmetre d'entrada i sortida), modifiqui la llista de manera que cada posició de la llista modificada contingui la suma dels elements que hi havia a la llista fins a aquella posició **abans** de ser modificada.

Per exemple, si tenim la llista

$$L = [1, 3, 1, 2, 3]$$

ha de modificar la llista de manera que quedi

$$L = [10, 9, 6, 5, 3]$$

Si ens hi fixem, a la primera posició hi ha la suma $1 + 3 + 1 + 2 + 3$, a la segona posició hi ha la suma $3 + 1 + 2 + 3$, a la tercera posició hi ha la suma $1 + 2 + 3$, a la quarta posició hi ha la suma $2 + 3$ i a la cinquena posició hi ha la suma 3.

La funció que verificarem és aquesta:

```

1  int llistaAcumulada (list<int>& l, list<int>::iterator it2)
2  // Pre: l = L ∧ l.begin() ≤ it ≤ l.end()
3  {
4      int r = 0
5      /// Cas base.
6      if (it2 == l.end())          r = 0;
7
8      else      /// not it2 != l.end()
9      {        /// Cas recursiu.
10         int x = *it2;
11         ++it2;
12         int r = llistaAcumulada(l, it2);
13         --it2;
14         *it2 = x + r;
15         r = x + r;
16     }
17
18     return r;
19
20 // Post:  $\left[ \bigvee_{it2 \leq it < l.end()} (*it = suma(L, it)) \right] \wedge r = suma(L, it2)$ 
21 };

```

$$\text{on } suma(L, it2) = \sum_{it2 \leq it < L.end()} (*it).$$

És a dir, la funció $suma(L, it2)$ retorna la suma dels elements que hi ha entre l'iterador it fins al final de la llista. Cal fixar-se en el fet que el paràmetre que li passem a aquesta funció és la llista L i no pas la llista l . Això és així perquè cal posar la suma dels elements de la llista *original*, i no pas de la llista que estarem modificant.

Veureu que aquesta funció té un paràmetre extra. Això vol dir que la cridarem amb una altra funció, que és aquesta:

```

1 void llistaAcumulada (list<int>& L)
2 {
3     int x = llistaAcumulada(L, L.begin());
4     return x;
5 };

```

De fet, aquest paràmetre extra té una explicació: una primera aproximació a una solució per a aquest problema seria, per exemple, calcular la suma que ens interessa amb la implementació de la funció `suma(L, it)`, i després fer els càlcul corresponent. Però si ho féssim així, veuríem que estariem recorrent la llista més vegades de les que són necessàries.

L'aproximació que fem servir en aquest exemple és la següent: si hem de calcular la postcondició sobre un tros de la llista que va de l'iterador `it2` fins al final de la llista, el que farem serà desar el valor que tenim a la posició `it2`, augmentar aquest iterador perquè tinguem el resultat calculat per a la resta de la llista (és a dir, per al tros que va del l'iterador `++it2` fins al final), i demanar a aquesta crida recursiva que ens torni la suma de la llista de tot els tros calculat, amb la qual cosa, ens estalviem la potencial crida a la implementació de la funció `suma`.

Procedim a fer la verificació per passos:

1. Cas base. La postcondició que cal complir és la següent:

$$\left[\bigvee_{it2 \leq it < l.end()} (*it = suma(L, it)) \right] \wedge r = suma(L, it2)$$

Tenint en compte que `it2 == l.end()` i que `r = 0`, ens queda que:

$$\left[\bigvee_{l.end() \leq it \leq l.end()} (*it = suma(L, it)) \right] \wedge r = 0$$

La primera part de l'invariant és trivialment certa, perquè és un interval buit, i la segona també perquè la suma d'un interval buit és el neutre de la suma, que és 0.

2. Les crides recursives es fan correctament. Això consisteix en: (i) veure que els paràmetres es poden avaluar sempre. Les operacions que potencialment poden avaluar a un error són `*it2` i `++it2` (generen un error quan és `it2 = l.end()`), però són en el cas recursiu i per tant, ens trobem sota la protecció `it2 != l.end()`, i (ii) verificar que es compleix amb la precondition de les crides recursives. En aquest cas cal verificar que el paràmetre `it2` es troba dins de l'interval, cosa que és certa per la precondition, més la condició de la crida recursiva (`it2 != l.end()`).
3. Després de les crides recursives, arribem a la postcondició. Aquí cal veure que després d'elles crides recursives tenim més codi, que és el que ens ajudarà a arribar a la postcondició. La postcondició de la crida recursiva (la hipòtesi d'inducció) és la següent:

$$\left[\bigvee_{it2+1 \leq it < l.end()} (*it = suma(L, it)) \right] \wedge r = suma(L, it2 + 1)$$

Ara cal veure que la postcondició que cal complir és la següent:

$$\left[\bigvee_{it2 \leq it < l.end()} (*it = suma(L, it)) \right] \wedge r = suma(L, it2)$$

La primera part la podem dividir en dues parts:

$$\left[\bigvee_{it2+1 \leq it < l.end()} (*it = suma(L, it)) \right] \wedge *it2 = suma(L, it2)$$

Fixem-nos que la primera part ja està garantida per la postcondició de la crida recursiva. Per tant, només hem de garantir que es compleix `*it2 = suma(L, it2)`. I això ho complim perquè a la línia 12 fem l'assignació `*it2 = x + r`. Tenint en compte que `x = *it2` (línia 10) i que `r = suma(L, it2+1)` (postcondició de la crida recursiva), llavors tenim que

$$*it2 = *it2 + suma(L, it2 + 1) = suma(L, it2)$$

Només ens queda verificar que complim la segona part de la postcondició:

$$r = suma(L, it2)$$

que ve garantit per la línia 15, ja que fem `r = x + r` que, com acabem d'explicar és equivalent a fer

$$*it2 = suma(L, it2)$$

4. Funció de fita. Agafem com a funció de fita la mida de la llista que va de `it2` fins al final. Com que quan fem la crida recursiva el paràmetre `it2` s'incrementa, això vol dir que la mida d'aquest tros de llista es decrementa. Simplement cal assegurar que l'increment `++it2` de la línia 11 no produeix cap error, i això ve garantit per la precondició i per la condició sota la qual executem la crida recursiva (`it2 != l.end()`).

Un petit comentari sobre el programa: possiblement, l'assignació que fem a la línia 10 no sigui necessari. De fet, és possible que la variable `x` no calgui.

4.5 Intercala Piles (Punters)

Siguin P i Q dues piles implementades amb punters. La seva estructura és:

```
1 private:
2
3     struct node
4     {
5         T        info;
6         node*    seguent;
7     };
8
9     node*    top;
10    int      mida;
```

Bàsicament, la implementació consisteix en una cadena d'instàncies de `struct node` enllaçats amb un sol punter. La classe conté el punter al cim de la pila (`top`) i també la `mida` (per no haver de recórrer tota la pila cada vegada que ho haguem de menester).

Cal programar el mètode `void intercala (Pila<T>& q)` en aquesta classe, de manera que si tenim les següents piles:

1	
2	
3	15
4	16
5	17
-----	-----
P	Q

la crida `P.intercala(Q)` faci que la pila P quedi així:

1
15
2
16
3
17
4
5

P

i la pila Q no es veu modificada. També cal assegurar que els nous elements de la pila P són **còpia** dels de Q , és a dir, que la pila P no té cap enllaç a cap element de la pila Q . Com a preconditionió tindrem que $|P| \geq |Q|$.

El codi que pot resoldre aquest problema pot ser aquest:

```
1 void intercala (Pila<T>& q)
2 {
3     node* punter_p = top;
```

```

4      node* punter_q = q.top;
5
6      while (punter_q != NULL)
7      {
8          node* nou = new node;
9          nou->info = punter_q->info;
10         nou->seguent = punter_p->seguent;
11         punter_p->seguent = nou;
12         punter_p = nou->seguent;
13         mida = mida + 1;
14
15         punter_q = punter_q->seguent;
16     }
17
18     return;
19 };

```

Ara cal definir la precondition i la postcondició. Quan tenim punter, podem fer servir diferents estratègies per a fer la pre/post (diferents formalismes). Aquí farem servir una notació que intentarà imitar la manera com estan els elements enllaçats en una estructura. Per exemple, per denotar la pila P farem això:

$$P = p_1 \rightarrow p_2 \rightarrow p_3 \rightarrow \cdots \rightarrow p_n$$

on p_1 és el cim de la pila. Per tant, la precondition ens queda així:

$$P = p_1 \rightarrow p_2 \rightarrow p_3 \rightarrow \cdots \rightarrow p_n$$

$$Q = q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow \cdots \rightarrow q_m$$

$$n \geq m$$

i la postcondició, així:

$$P = p_1 \rightarrow q_1 \rightarrow p_2 \rightarrow q_2 \cdots \rightarrow p_m \rightarrow q_m \rightarrow p_{m+1} \rightarrow p_{m+2} \cdots \rightarrow p_n$$

$$Q = q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow \cdots \rightarrow q_m$$

Pensem que cal que especifiquem també a la postcondició que no toquem la cua Q . Per altra banda, el fet que la variable `mida` hagi de tenir sempre el nombre d'elements de la llista apuntada per `top` estarà especificat a l'invariant de la classe.

D'aquesta manera, el codi complet queda així:

```

1  PRE:  P = p1 → p2 → p3 → ⋯ → pn
2        Q = q1 → q2 → q3 → ⋯ → qm
3        n ≥ m
4
5  void intercala (Pila<T>& q)
6  {
7      node* punter_p = top;
8      node* punter_q = q.top;
9
10     while (punter_q != NULL)

```

```

11      {
12          node* nou = new node;
13          nou->info = punter_q->info;
14          nou->seguent = punter_p->seguent;
15          punter_p->seguent = nou;
16          punter_p = nou->seguent;
17          mida = mida + 1;
18
19          punter_q = punter_q->seguent;
20      }
21
22      return;
23  };
24
25  POST:  $P = p_1 \rightarrow q_1 \rightarrow p_2 \rightarrow q_2 \dots \rightarrow p_m \rightarrow q_m \rightarrow p_{m+1} \rightarrow p_{m+2} \dots \rightarrow p_n$ 
26         $Q = q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow \dots \rightarrow q_m$ 

```

Per altra banda, tot i que no està explícit a la pre/post (caldrà posar-ho com a condició a la post) cal tenir en compte que totes dues piles no estan enllaçades.

Ara ja només ens falta proposar un invariant que, com en un gran nombre de casos, serà el debilitament de la postcondició. Però abans, mirem-nos la postcondició, i mirem què ens està dient: la pila **P** tindrà els elements de la pila **Q** entrellaçats a partir del cim. Què és el que hauria de dir l'invariant? Doncs que hem calculat el resultat... fins a un punt en concret de l'estructura que estem recorrent, en aquest cas, totes dues piles. Fixem-nos que el que fa el bucle, com en la majoria de casos amb què ens trobarem en algorismes d'aquest tipus, és que a cada pas de la iteració, avançarem un pas cap al resultat final. Per tant, un invariant que expressés això podria ser:

$$\begin{aligned}
 P &= p_1 \rightarrow q_1 \rightarrow p_2 \rightarrow q_2 \dots \rightarrow p_{k-1} \rightarrow q_{k-1} \rightarrow p_k \rightarrow p_{k+1} \dots \rightarrow p_n \\
 punter_p &= p_k \wedge punter_q = q_k \wedge k \leq m \\
 Q &= q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow \dots \rightarrow q_m
 \end{aligned}$$

Cal veure que en aquest invariant també hi apareixen les variables **punter_p** i **punter_q**, que són a l'invariant. Les altres variables són auxiliars i no les haurem de menester a l'invariant.

Procedim a fer la verificació per passos:

1. Precondició + inicialitzacions $\Rightarrow$ INV. En aquest cas, com que tenim que **punter_p** = **p.top** i **punter_q** = **q.top**, llavors l'invariant és

$$\begin{aligned}
 P &= p_1 \rightarrow p_2 \dots \rightarrow p_n \\
 punter_p &= p_1 \wedge punter_q = q_1 \wedge k \leq m \\
 Q &= q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow \dots \rightarrow q_m
 \end{aligned}$$

Fixem-nos que, llevat de $punter_p = p_1 \wedge punter_q = q_1$, la resta és, precisament, la precondició, ja que l'interval que hi ha **anterior** a p_1 i a q_1 és buit.

2. Condió del bucle + INV $\Rightarrow$ INV.

Cal veure què passa si, partint de l'invariant (hipòtesi d'inducció) fem una iteració (marquem en negreta la part que **ja ha estat tractada**).

$$P = p_1 \rightarrow q_1 \rightarrow p_2 \rightarrow q_2 \cdots \rightarrow p_{k-1} \rightarrow q_{k-1} \rightarrow p_k \rightarrow p_{k+1} \cdots \rightarrow p_n$$

$$punter_p = p_k \wedge punter_q = q_k \wedge k \leq m$$

$$Q = q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow \cdots \rightarrow q_m$$

Ara cal veure què passa amb la part principal de l'invariant. Si fem

```
node* nou = new node;
nou-> info = punter_q->info;
```

tindrem que la variable **nou** és una còpia de q_k .

La instrucció **nou->seguent = punter_p->seguent;** enllaça $q_k \rightarrow p_{k+1}$ (recordem que *punter_p* apunta a l'element que nosaltres anomenem p_k i, per tant, *punter_p* -> *seguent* apunta a p_{k+1}). Finalment, la instrucció **punter_p->seguent = nou;** enllaça $p_k \rightarrow q_k$. Per tant, ens queda:

$$P = p_1 \rightarrow q_1 \rightarrow p_2 \rightarrow q_2 \cdots \rightarrow p_{k-1} \rightarrow q_{k-1} \rightarrow p_k \rightarrow q_k \rightarrow p_{k+1} \cdots \rightarrow p_n$$

Ara mirem què passa amb la part de l'invariant que parla de les variables **punter_p** i **punter_q**. Com que fem **punter_q = punter_q->seguent;** tenim que *punter_q* = q_{k+1} o *punter_q* = *NULL*, i com que fem

```
nou->seguent = punter_p->seguent;
punter_p = nou->seguent;
```

és a dir:

```
punter_p = nou->seguent = punter_p->seguent;
```

també tenim que *punter_p* = p_{k+1} .

La part que té a veure amb **Q** roman igual, ja que només hem fet una còpia d'un element de **Q**, però no l'hem modificat. Per tant, després d'una iteració tenim

$$P = p_1 \rightarrow q_1 \rightarrow p_2 \rightarrow q_2 \cdots \rightarrow p_{k-1} \rightarrow q_{k-1} \rightarrow p_k \rightarrow q_k \rightarrow p_{k+1} \cdots \rightarrow p_n$$

$$punter_p = p_{k+1} \wedge punter_q = q_{k+1} \wedge k \leq m$$

$$Q = q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow \cdots \rightarrow q_m$$

que és el nostre invariant, ara havent avançat una posició.

3. Negació de la condició + $INV \Rightarrow POST$.

Quan acabem el bucle, tenim que `punter_q = NULL`, cosa que, segons la manera com hem caracteritzat nosaltres la pila, voldrà dir que $k = m + 1$, és a dir: `punter_p = p_{m+1}`. Això voldrà dir que l'invariant serà:

$$P = p_1 \rightarrow q_1 \rightarrow p_2 \rightarrow q_2 \cdots \rightarrow p_m \rightarrow q_m \rightarrow p_{m+1} \rightarrow p_{m+2} \cdots \rightarrow p_n$$

$$punter_p = p_{k+1} \wedge punter_q = NULL \wedge k \leq m$$

$$Q = q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow \cdots \rightarrow q_m$$

que és, precisament, la postcondició de la funció.

4. Funció fita. En aquest cas, la funció fita pot ser $m - k$. Fixem-nos que a cada pas k és una unitat més gran (cal tenir en compte que estem parlant de la caracterització que nosaltres hem fet de les piles), ja que a cada pas tenim que `p_punter` passa de p_k a p_{k+1} (igual amb `punter_q`). A més, com que l'invariant ens diu que $k \leq m$, sempre tenim que aquesta funció estarà definida sobre els naturals. De manera més genèrica, la funció de fita seria la mida de la part que ens queda per processar, que en aquest cas és la part de la pila `Q` que encara no hem processat. Amb la caracterització que hem fet nosaltres de les piles, això és, com hem dit, $m - k$.

5 Algoritmes Recursius

5.1 Fulles Esquerres

Les **fulles** d'un arbre binari són els nodes que no tenen ni fill dret ni fill esquerre. Una **fulla esquerra** és una fulla que és el fill esquerre del seu pare. Cal fer una funció **recursiva** tal que, donat un arbre binari, torni el nombre de fulles esquerraes que té l'arbre.

Per a resoldre aquest problema, proposem el següent programa:

```

1  int fullesEsquerres (arbreBin<int> A, char p)
2  {
3  // PRE: no buit(A)
4
5      int r = 0;
6
7      /// Cas base
8      if (A.fe().es_buit() and A.fd().es_buit())
9      {
10         if (p == 'e')    r = 1;
11         else              r = 0;
12     }
13     else{
14         int r = 0;
15         if (not A.fe().es_buit()) r = fullesEsquerres(A.fe(), 'e');
16         if (not A.fd().es_buit()) r += fullesEsquerres(A.fd(), 'd');

```



```

17     }
18
19     return r;
20 // POST: r = |\{f \in A \mid fillEsquerre(f)\}|
21 };
22
23 int fullesEsquerres (arbreBin<int> a)
24 {
25     /// PRE: not a.es_buit()
26
27     int r = 0;
28     if (not a.fe().es_buit()) r = fullesEsquerres(a.fe(), 'e');
29     if (not a.fd().es_buit()) r += fullesEsquerres(a.fd(), 'd');
30
31     return r;
32 };

```

Fixem-nos en diferents detalls, per exemple, en aquest cas la funció que calcula el problema fa una crida a una altra funció (que és qui fa realment el càlcul del nombre de fulles esquerres. Això vol dir que si l'arbre que passem a la primera funció és un arbre que no té cap fill, la funció retornarà un zero. Aquesta decisió és arbitrària, i depèn de com estigui redactat l'enunciat. Per altra banda, cal que tinguem en compte la definició de fulla esquerra: que no tingui cap fill i que, a més a més, sigui el fill esquerre del seu pare. A la postcondició hem fet servir aquesta notació:

$$fillEsquerre(f)$$

que és una funció que torna **cert** si i només si **f** és una fulla esquerra. Aquesta funció és una manera de simplificar la notació, no és cap funció auxiliar ni cal implementar-la (recordeu que de vegades fem servir altres funcions a les especificacions, per exemple, la funció mida d'un arbre: $|A|$ per a referir-nos al nombre de nodes que té un arbre).

Verificarem la segona funció (`int fullesEsquerres (arbreBin<int> A, char p)`) i prou, ja que la verificació de la funció principal és directa.

Abans de començar la verificació, cal que tinguem una cosa en compte:

$$lletra = 'e' \Leftrightarrow fe(pare(A)) = A$$

És a dir, un node és fill esquerre si i només si el paràmetre **p** que rep és la constant **'e'**. Això és així perquè des de la primera crida, només passem aquesta constant als fills esquerres, mentre que als drets els passem el paràmetre **'d'**. Per tant, un node serà fulla esquerra si i només si $lletra = 'e'$ i a més, no té fills.

Procedim a fer la verificació per passos:

1. Cas base. Com que la precondition diu que l'arbre no pot ser buit, llavors tenim un sol cas: quan el cas no té fills. En aquest cas, si és un fill esquerre ($lletra = 'e'$) l'arbre tindrà una fulla esquerra, que és el mateix node. Altrament, no en tindrà cap (serà fulla, però dreta). Amb les

assignacions que fem a les línies 10 i 11 verifiquem que fem el càlcul segons la postcondició.

2. Les crides recursives es fan correctament. Això consisteix en: (i) veure que els paràmetres es poden avaluar sempre. En aquest cas, cal verificar que totes les funcions parcials es poden avaluar sense donar error. Concretament cal verificar que les següents expressions són avaluables: `A.fe().es_buit()`, `A.fe()`, `A.fd().es_buit()`, `A.fd()`. En aquest cas, no hi ha error perquè la precondició ens diu que `A` no és buit, i (ii) verificar que es compleix amb la precondició de les crides recursives. En aquest cas, cal veure que només fem la crida si el paràmetre (`A.fe()`) no és buit: `if (not A.fe().es_buit())` (ídem amb el fill dret).
3. Després de les crides recursives, arribem a la postcondició. Cal tenir en compte que si ens trobem en el cas recursiu, és perquè el node actual (`A`) no és una fulla. Per tant, el nombre de fulles esquerres que tindrà serà les que té al fill esquerre més les que té al fill dret. Com que les postcondicions de les crides recursives són $r = |\{f \in A.fe() \mid fillEsquerre(f)\}|$ i $r = |\{f \in A.fd() \mid fillEsquerre(f)\}|$, i el que fem és justament sumar-les totes dues a la variable `r` tenim que aquesta variable conté el nombre de fulles esquerres del fill dret **més** les del fill dret, cosa que és exactament el que demana la postcondició.
4. Funció de fita. En aquest cas, podem fer servir la funció `alçada(A)`, que ens torna un natural. Ja sabem que `alçada(A) = alçada(A.fe()) + 1`, és a dir, decreix estrictament a cada crida recursiva en el nostre programa, quan `A.fe()` està ben definit (ídem fill dret). En aquest cas, cal recordar que fem les crides recursives (com hem explicat a l'apartat anterior) únicament quan el paràmetre no és buit.

5.2 Reordena Pila

Una pila **quasi-ordenada** és una pila (d'enters) que té tots els elements en ordre (el més petit al fons de la pila) **llevat de dos elements** que estan intercanviats i a més són **sempre** un a sobre de l'altre a la pila.

Per exemple, la pila

```
| 9 |  
| 7 | <-  
| 8 | <-  
| 5 |  
| 3 |  
| 2 |  
-----
```

està quasi-ordenada, ja que té els elements 7 i 8 únicament desordenats.

Cal verificar la funció `void reordenaPila (stack<int> &P)` tal que, donada una pila quasi-ordenada d'enters positius, retorna la mateixa pila, però ordenada. Cal tenir en compte que el paràmetre `P` és d'entrada i sortida.

```
1 void reordenaPila (stack<int> &P)  
2 // Pre: no buida(P) i (si |P| > 1 => P pila semi-ordenada).  
3 {  
4     int i = P.top();  
5     P.pop();  
6  
7     if (P.empty()) P.push(i); /// Cas base.  
8  
9     else if (i < P.top())      /// Cas base.  
10    {  
11        int j = P.top();  
12        P.pop();  
13        P.push(i);  
14        P.push(j);  
15    }  
16  
17    else                        /// Cas recursiu.  
18    {  
19        reordenaPila(P);  
20        P.push(i);  
21    }  
22  
23    return;  
24 // Post: P ordenada.  
25 };
```

Si ens hi fixem, hi ha 2 casos base: un que serveix per a una pila d'un sol element, i un altre que és quan trobem el parell d'elements que no estan ordenats.

Procedim a fer la verificació per passos:

1. Cas base. Hi ha dos casos base. El primer és quan la pila té un sol element. Fixem-nos que ho sabem perquè prèviament hem desempilat un element

de la pila i ara tenim la pila buida. Òbviament, una pila d'un sol element està ordenada, per tant, reempilem l'element que havíem desempilat i la pila no pateix cap canvi.

El segon cas base és quan la pila té més d'un element i trobem els dos elements desordenats al cim de la pila. En aquest cas, els reordenem i acabem. Com que sabem que la pila és semi-ordenada (perquè té més d'un element) llavors la resta de la pila està ordenada (per definició de semi-ordenat).

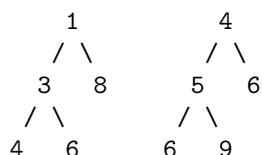
2. Les crides recursives es fan correctament. Això consisteix en: (i) veure que els paràmetres es poden avaluar sempre. En aquest cas, la pila `P` es pot avaluar perquè és la pila que hem rebut com a paràmetre, amb un element desempilat. Cal assenyalar que hem pogut desempilar-lo perquè la precondition indica que `P` no és buida. Per tant, el paràmetre es pot avaluar sense error. (ii) verificar que es compleix amb la precondition de les crides recursives. En aquest cas, sabem que la pila no és buida, i a més, com que els dos elements del cim no són els que estan ordenats (altrament hauríem entrar a dins de la condició anterior) la pila que li enviem serà encara semi-ordenada. Penseu que per a complir la precondition cal que la pila no sigui buida, i després cal que si té més d'un element (com és el cas) llavors sigui semi-ordenada. Com que sabem per la precondition que és semi-ordenada, i sabem que els elements desordenats no són els dos del cim, llavors sabem també que la pila sense el cim és semi-ordenada.
3. Després de les crides recursives, arribem a la postcondició. La postcondició de la crida recursiva (la hipòtesi d'inducció) és `P.pop()` ordenada. Per tant, com que sabem que l'element que hem tret de la pila estava també ordenat, si el posem a la pila, estarà en orde amb els mateixos elements que hi havien a la pila.
4. Funció de fita. En aquest cas és fàcil veure que agafem `alçada(P)` com a funció de fita: torna un nombre natural, i a més decreix estrictament a cada crida recursiva. Recordem que podem fer `P.pop()` perquè la precondition indica que la pila no és buida.

5.3 Arbre Creixent

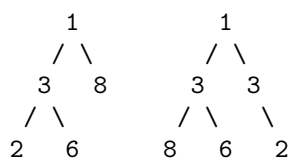
Cal fer una funció tal que, donat un arbre binari, torni **true** si i només si és un **arbre creixent**.

Un arbre és **creixent** si i només si per a tots els nodes de l'arbre es compleix aquesta propietat: el seu valor és **estrictament inferior** a tots els valors que hi ha en els nodes que en pengen.

Per exemple, aquest dos arbres són creixents



En canvi, aquests dos no ho són:



El problema admet una solució senzilla, i força intuïtiva. El que cal veure és com en fem l'especificació. En un principi, veiem que un arbre és creixent si per a tot node de l'arbre, es compleix que és estrictament inferior a tots els nodes *que té per sota*. Això podria ser una idea per a escriure una postcondició. Però si ho refinem una mica més, veurem que podrem fer una postcondició més compacta, que ens ajudarà a escriure el programa i a verificar-lo. Per exemple, podem aprofitar la *transitivitat* de l'ordre per a definir aquesta condició: **l'arrel és estrictament més petita que l'arrel del seu fill esquerre i que l'arrel del seu fill dret, i aquesta condició l'exigim a tots els altres nodes de l'arbre**. Complirà aquesta condició amb allò que demana l'enunciat? Anem a veure què això és cert, és a dir, que l'enunciat és cert si i només si es compleix aquesta condició.

Agafem un arbre qualsevol, amb arrel a , i fixem qualsevol node de l'arbre n . Assumim que es compleix la condició que hem esmentat. Això vol dir que des de l'arrel a fins al node n hi ha un camí: $a, a_1, a_2, \dots, n$. Com que és un camí, això vol dir que el node a_i és el **pare** del node a_{i+1} , i com que es compleix la condició, això vol dir que $a < a_1 < a_2 < \dots < n$. Això ho fem per a l'arrel i per a qualsevol node n .

Ara demostrem el *i només si*: Seguim amb els dos nodes i el mateix camí $a, a_1, a_2, \dots, n$. Si no es compleix la condició llavors això vol dir que hi ha almenys un parell de nodes adjacents tals que $a < a_1 < a_2 < \dots < a_i \geq a_{i+1} < \dots < n$. Però això precisament vol dir que l'arbre no és creixent.

Per tant, ja tenim una propietat més simple i compacta per a especificar la nostra solució.

```

1  bool arbreCreixent (arbreBin<int> A)
2  // Pre: cert.
3  {
4      bool r = false;
5
6      if (A.es_buit()) r = true; // Cas base
7
8      else if (A.fe().es_buit() and A.fd().es_buit())
9          r = true; // Cas base
10
11     else{ // not A.es_buit() and (A.fe().no_buit() or A.fd().no_buit())
12         // cas recursiu
13
14         r = true;
15
16         if (not A.fe().es_buit())
17             r = A.arrel() < A.fe().arrel() and arbreCreixent(A.fe());
18
19         if (not A.fd().es_buit() and r)
20             r = A.arrel() < A.fd().arrel() and arbreCreixent(A.fd());
21     }
22
23     return r;
24 };
25 // Post: r = A arbre creixent
26 };

```

Procedim a fer la verificació per passos:

1. Cas base. En el primer cas base ens trobem amb un arbre buit, que és trivialment creixent perquè no té cap element. En el segon cas base ens trobem un arbre sense fills, en què la condició també es compleix de manera trivial perquè no té cap element amb què comparar-se.
2. Les crides recursives es fan correctament. Això consisteix en: (i) veure que els paràmetres es poden avaluar sempre. En aquest cas, les operacions potencialment problemàtiques són `A.fe().es_buit()` i `A.fd().es_buit()` `A.fe()`, `A.fd()`, `A.fe().arrel()` i `A.fd().arrel()` però com que es fan sota la protecció `not A.es_buit()` llavors estan definides.
(ii) verificar que es compleix amb la precondition de les crides recursives. Com que la precondition és trivial, es compleix.
3. Després de les crides recursives, arribem a la postcondició. La postcondició de totes dues crides recursives ens diuen que `r` serà cert si i només si tots dos fills (si existeixen) són arbres creixents, i, a més, l'arrel de l'arbre és estrictament més petit que les arrels de tots dos subarbres (esquerra i dret). I això, com ja hem raonat anteriorment, és la definició d'arbre creixent.
4. Funció de fita. En aquest cas podem fer servir l'alçada o el nombre d'elements de l'arbre. Simplement cal veure que, si l'arbre no és buit, llavors `alçada(A) = alçada(A.fe()) + 1`. Aquesta equació està definida

si A és un arbre no nul, cosa que es verifica per la protecció de la condició de la crida recursiva.

5.4 Llista Ordenada per Paritat

Cal fer una funció **recursiva** tal que, donada una llista l d'enters **estrictament més grans que zero**, torni **true** si i només si la llista està ordenada **per paritat**. Diem que **una llista està ordenada per paritat** quan els nombres d'una mateixa paritat (és a dir: senars per una banda i parells per una altra) estan ordenats de manera **estrictament creixent**. Per exemple, la llista:

7 9 4 33 6 8 22 57 32 34

està ordenada per paritat, ja que els parells (4 6 8 22 32 34) estan ordenats de manera estrictament creixent, i els senars (7 9 33 57) també. En canvi, la llista:

7 9 4 33 6 8 23 57 32 34

no ho està per culpa del 23.

Aquesta funció, sense el detall de l'ordenació per paritat, no tindria massa problemes. Però justament l'aparició de la paritat és el que ho fa complicat.

Hi podrien haver diferents alternatives per a resoldre el problema: una seria dividir la llista en dues parts: la dels senars i la dels parells, i enviar-ho a la funció que comprova si una llista està ordenada, però ja us podeu imaginar que no seria possiblement la millor solució⁴.

De tota manera, es tracta de veure què podem aprofitar de la versió *simple* de la verificació de l'ordre d'una llista per a aquest problema (per simple, volem dir el problema de saber si una llista està ordenada). Fixem-nos que si vulguéssim fer una funció (recursiva) que comprova si una llista està ordenada, la solució seria bastant simple: per a cada element, mirariem si es compleix una propietat (la de ser més gran o igual que el seu predecessor). Si es compleix aquesta condició, faríem la crida recursiva per saber si es compleix sobre la resta de la llista, altrament, tornariem fals. De fet, seria un esquema simple de cerca implementat recursivament. Hauríem de mirar alguna manera de fer que aquest mateix esquema es pogués aplicar en aquest cas, per exemple, pensant una propietat per a qualsevol element de la llista. Aquesta propietat seria la de ser més gran (o igual) que la resta dels elements anteriors de la llista... de la mateixa paritat. El problema és que per saber si és més gran que tots els elements anteriors de la llista de la mateixa paritat, caldria recórrer tota aquesta part anterior una vegada per a cada element, i, com us podeu imaginar, això no és ni de bon tros eficient. Però per altra banda, cal que pensem que no caldria comparar l'element actual amb tots els elements anteriors (de la mateixa paritat), sinó amb el **màxim** d'aquells elements! És cert que per calcular el màxim dels elements anteriors també caldria recórrer tot l'interval. Llevat, és clar, que ens ho passessin com a paràmetre de la funció. I aquí és quan fem una immersió de paràmetres:

⁴De tota manera, asimptòticament, aquesta solució seria tan eficient com la que acabarem proposant.


```

1 bool auxiliar (const list<int> &l, iterador it, int parell, int senar)
2 // Pre: la llista l ordenada per paritat des de l.begin() fins a it-1
3 //      parell maxim parell de la llista l[begin() ... it-1]
4 //      senar maxim senar de la llista l[begin() ... it-1]
5 {
6     bool b = true;
7
8     if (it == l.end()) b = true;          // Cas base 1.
9
10    else
11    {
12        if (*it % 2 == 0)
13        {
14            if (*it <= parell) b = false;    // Cas base 2a
15            else b = auxiliar(l, ++it, *it, senar); // Cas rec 1a
16        }
17        else
18        {
19            if (*it <= senar) b = false;    // Cas base 2b
20            else b = auxiliar(l, ++it, parell, *it); // Cas rec 1b
21        }
22    }
23
24    return b;
25 };
26 // Post: b = l ordenada per paritat
27
28 bool ordenadaParitat (const list<int> &l)
29 {
30     iterador it = l.begin();
31     if (it == l.end()) return true; /// per si llista buida.
32
33     if (*it % 2 == 0) return auxiliar (l, ++it, *it, 0 );
34     else return auxiliar (l, ++it, 0, *it);
35 };

```

Fixem-nos que, com que fem immersió (de paràmetres) necessitem fer una funció que serà cridada per la funció que ens han demanat (que en aquest cas farà simplement d'interfície).

Procedim a fer la verificació per passos, bo i recordant que hem reduït el problema de saber si un element està ordenat per paritat en una llista a saber si és més gran o igual que el màxim element de la mateixa paritat en la part anterior de la llista. A la verificació, quina diem que la llista està ordenada voldrà dir que està ordenada per paritat:

1. Cas base. En el cas base1 tenim que la precindició ens diu que tota la llista anterior a `it` està ordenat, i com que `it = l.end()` vol dir que la llista està tota ordenada, cas en què tornem cert i complim la postcondició. Els casos base 2a i 2b (són simètrics) comparen l'element actual amb el màxim de la seva paritat: si és parell amb la variable `parell`, si és senar, amb la variable `senar`. Pel que hem discutit anteriorment, això vol dir que la llista no està ordenada, i tornem fals, com demana la postcondició.
2. Les crides recursives es fan correctament. Això consisteix en: (i) veure

que els paràmetres es poden avaluar sempre. Les úniques expressions que ens poden donar problemes són `*it` i `it++`, que no estan definides quan l'iterador és `l.end()`. Però això no pot passar perquè ens trobem en el cas contrari.

(ii) verificar que es compleix amb la precondition de les crides recursives. La gran part de la discussió de la verificació d'aquest algoritme cau en aquesta part, perquè és on fem els càlculs substancials. Hi ha dos casos, que són simètrics: cas recursiu 1a i 1b. En discutirem un només, perquè tot el que direm ens servirà per a l'altre. Assumim que `*it` és parell. Per saber si està ordenat, llavors caldrà veure si és més gran que el màxim dels parells de la part anterior de la llista (això és el que hem discutit abans de fer la verificació). Per la precondition sabem que aquest màxim es troba a la variable `parell`. Si està ordenat (línia 17) llavors fem la crida recursiva. Per a la crida recursiva cal assegurar que els paràmetres que passem són correctes, és a dir, que compleixen la precondition (aplicada als paràmetres!):

- (a) la llista `l` ordenada per paritat des de `l.begin()` fins a `it`. Això és cert perquè ja hem vist que la precondition ens diu que la llista estava ordenada fins a l'iterador `it-1`, i ara, ja que es compleix que `*it > parell` sabem que la llista està ordenada fins a `it`.
 - (b) `parell` és el màxim parell de la llista `l[begin() ... it]`. Com que `*it > parell` i `parell` (per la precondition) és el màxim de la llista fins a `it-1`, llavors `*it` és el màxim de la llista fins a `it`.
 - (c) `senar` és el màxim senar de la llista `l[begin() ... it]`. Com que `*it` és parell, el senar més gran fins a `it` segueix sent `senar`.
3. Després de les crides recursives, arribem a la postcondició. La postcondició de la crida recursiva és la mateixa que la de la funció.
 4. Funció de fita. En aquest cas, agafem la mida de la llista des de `it` fins al final. Sempre torna un natural, i només cal veure que si $it \neq l.end()$, llavors $|l[it..end())| = 1 + |l[it + ..end())|$.

5.5 Pila Ordenada

Cal escriure i verificar una funció **recursiva** tal que, donada una pila d'enters positius, no necessàriament ordenada, retorni la mateixa pila però eliminant-ne els elements que no estiguin ordenats de manera **extrictament creixent** (des del cim cap al fons de la pila). Per exemple, si tenim la pila:

```
| 3 |  
| 5 |  
| 4 |  
| 5 |  
| 7 |  
| 8 |  
| 2 |  
-----
```

veiem que si eliminem el 4, el 2 i el segon 5, llavors la pila estarà ordenada:

```
| 3 |  
| 5 |  
| 7 |  
| 8 |  
-----
```

És a dir, a partir del cim, anem eliminant els element que no estiguin ordenats. La pila ha de quedar ordenada de manera creixent a partir del cim (de dalt a baix, anem a dir).

Per a plantejar una solució per a aquest problema, cal veure que si tenim una pila, cal decidir si hem d'empilar el cim de la pila o no, i que aquesta decisió depèn de l'element que tenim *a sobre*. Fixem-nos que això de dir *eliminar els element que no estiguin ordenats* en realitat vol dir *eliminar els elements tals que siguin menors al que tenen a sobre*. És cert que hi podrien haver altres alternatives, per exemple, eliminar l'element que es troba *a sota de la pila* si és menor al cim de la pila, però això implica actuar sobre dos elements de la pila alhora. Si prenem la primera idea, veurem que per saber si el cim de la pila ha de ser eliminat o no, només ens cal saber l'element que té a sobre. Òbviament, això implica que cal passar aquest element com a paràmetre extra de la funció, i que el cim de la pila sempre hi serà. Per això la funció que cridarem simplement treu el cim de la pila, crida a la funció recursiva que és qui farà tot el càlcul, i després torna a empilar el cim que tenia. La funció queda d'aquesta manera:

```

1  stack<int> auxiliar (stack<int> P, int i)
2  // Pre: cert
3  {
4      if (P.empty()) { // Cas base: no fa res };
5
6      else // not P.empty()
7      { // Cas recursiu
8          int ii = P.top();
9          P.pop();
10
11          P = auxiliar (P,max(ii,i));
12          if (ii > i) P.push(ii);
13      }
14
15      return P;
16  // Post:  $\forall_{0 \leq j < P.size()} P[j] < P[j+1]$ , on  $P[0]$  es el cim,
17           P[1] l'element que te a sota, etc,
18           i tots els elements de P son mes grans que i.
19  };
20
21  stack<int> pilaOrdenada (stack<int> P)
22  {
23      int i = P.top();
24      P.pop();
25      P = auxiliar (P,i);
26      P.push(i);
27      return P;
28  };

```

La postcondició de la funció que verificarem està explicada com si la pila fos un vector, perquè si l'haguéssim d'explicar com a pila, ho hauríem de fer de manera equacional:

$$pilaOrdenada(empila(x, P), y) = \begin{cases} empila(x, pilaOrdenada(P, x)) & \text{si } x > y \\ pilaOrdenada(P, x) & \text{si } x \leq y \end{cases}$$

(i també: $pilaOrdenada(p_buida, y) = p_buida$).

Procedim a fer la verificació per passos:

1. Cas base. La pila és buida, i per tant, compleix de manera trivial la postcondició.
2. Les crides recursives es fan correctament. Això consisteix en: (i) veure que els paràmetres es poden avaluar sempre. Potencialment, les expressions que poden portar problemes són `P.pop()` i `P.top()`, que no són totals. Per sempre s'executen sota la protecció `not P.empty()`.
(ii) verificar que es compleix amb la precondition de les crides recursives. La precondition és trivial.
3. Després de les crides recursives, arribem a la postcondició. La postcondició de la crida recursiva (la hipòtesi d'inducció) ens diu que $\forall_{0 \leq j < P.pop().size()} P[j] <$

$P[j + 1]$ i tots els elements de $P.pop()$ son mes grans que $max(i, P.top())$. És a dir, la pila que torna la crida recursiva està ordenada i, a més a més, tots els elements són més grans que $max(i, P.top())$. Hi ha dues possibilitats: $i > P.top()$, cas en què els elements de la pila que ha retornat la crida recursiva compleixen la postcondició perquè tots els valors són més grans que i . $i \leq P.top()$, cas en què la pila que retorna la crida recursiva també compleix la postcondició, perquè tots els elements són més grans que $P.top()$ i, per transitivitat, que i . A més, cal fixar-se que en el segon cas, tots els elements de la pila són més grans que $P.top()$, (valor que hem eliminat de la pila a la línia 9 abans de fer la crida recursiva) és a dir, que si en aquest cas (línia 12) recuperem $P.top()$ a la pila, seguirem complint la postcondició. En canvi, en el primer cas, llavors $P.top()$ no pot anar a la pila perquè no compliria la postcondició ($i > P.top()$). Però justament en aquest cas, no posem $P.top()$ a la pila.

4. Funció de fita. En aquest cas, fem servir $|P|$. Ja sabem que si la pila no és buida (cosa que garantim per la condició de l'alternativ en el cas recursiu) tenim que $|P| = 1 + |P.pop()|$.

5.6 Inserció en Arbre de Cerca (Punters)

Donat un arbre binari de cerca i un element, cal fer un mètode que insereixi l'element en aquest arbre. La implementació de l'arbre és de la següent manera:

```

1  private:
2
3  struct node {
4      node* fe;
5      node* fd;
6      V      valor;
7  };
8
9  node* arrel;
```

Fixem-nos en què V és un tipus paramètric, del qual assumirem que és comparable amb els operadors $\leq$ i $\geq$. El mètode serà `insereix(const V& v)`, amb la següent pre/post:

```

1  PRE:  $v \notin PI$ .
2  void insereix(const V& v)
3  POST:  $arbreCerca(PI) \wedge v \in PI$ .
```

on PI és el paràmetre implícit. Hem comès un abús de notació amb $v \in PI$, que vol dir simplement que l'element v es troba com a element de l'arbre PI . Per altra banda, cal que recordem la definició de `arbreCerca`:

$$\begin{aligned}
 arbreCerca(aBuit) &= cert \\
 arbreCerca(plantar(x, aBuit, aBuit)) &= cert \\
 arbreCerca(plantar(x, aBuit, fd)) &= x \leq arrel(fd) \wedge arbreCerca(fd) \\
 arbreCerca(plantar(x, fe, aBuit)) &= arrel(fe) \leq x \wedge arbreCerca(fe) \\
 arbreCerca(plantar(x, fe, fd)) &= arrel(fe) \leq x \leq arrel(fd) \\
 &\quad \wedge arbreCerca(fe) \wedge arbreCerca(fd)
 \end{aligned}$$

Aquesta definició ens caldrà per verificar el mètode. El que proposem és un mètode que comprova si l'arbre (PI) és buit o no. Si ho és, afegeix l'element nou i torna, altrament, crida a un mètode que rep com a paràmetre un punter de tipus `node*` i l'element v .

```

1 PRE:  $v \notin PI \wedge PI = A$ .
2 void insereix(const V& v)
3 {
4     node* n = arrel;
5
6     if (n != NULL)
7         insereix(v,n);
8
9     else {
10        n = new node;
11        n->valor = v;
12        arrel = n;
13    }
14
15    return;
16 }
17 POST:  $arbreCerca(PI) \wedge v \in PI \wedge subArbre(A, PI) \wedge |A| + 1 = |PI|$ .

```

Primer de tot, caldria mencionar que la part de la postcondició

$$subArbre(A, PI) \wedge |A| + 1 = |PI|$$

té com a única missió assegurar que el PI és l'arbre que teníem al principi més l'addició del nou element (que la precondition assegura que no hi era). Altra-ment, si la funció modifiqués el PI de manera que contingués un sol node amb l'element n , ja compliria la portcondició $arbreCerca(PI) \wedge v \in PI$. Recordem que $subArbre(A, PI)$ és cert quan l'arbre A és un subarbre de PI , és a dir, que si *a sobre* de l'arbre PI hi posem A llavors coincideixen (mireu la llista d'exercicis i veureu que aquest problema hi és). A més, $v \in PI \wedge |A| + 1 = |PI|$ ens asseguren que l'únic element que hi hem posat és v .

Aquesta funció és fàcil de verificar en el cas que anem per la part de l'**else**, ja que amb la instrucció $n = \text{new}$; i $n \rightarrow \text{valor} = v$; ens assegurem que $n \in PI$, i amb la instrucció $\text{arrel} = n$; tenim un arbre amb un sol element, que és trivial per la segona equació de la definició de *arbreCerca*. En cas que anem per l'**if**, podeu verificar vosaltres mateixos que complirem la precondition d'aquest nou mètode i que la postcondició del nou mètode és la postcondició d'aquest mètode.

Ara anem pel nou mètode, que és el que porta la part del càlcul en cas que l'arbre no sigui buit. El mètode que proposem és aquest:

```

1 PRE:  $v \notin PI \wedge \text{notbuit}(PI) \wedge PI = A$ .
2 void insereix(const  $\forall x \ v$ , node* n)
3 {
4     node* pivot;
5
6     if (n->valor < v) {
7         if (n->fd != NULL)
8             insereix(v, n->fd);
9         else {
10            node* nn = new node;
11            nn->valor = v;
12            n->fd = nn;
13        }
14    }
15    else {
16        // n->valor > v
17        // isomorf al cas anterior.
18    }
19
20    return;
21 };
22 POST:  $\text{arbreCerca}(PI) \wedge v \in PI \wedge \text{subArbre}(A, PI) \wedge |A| + 1 = |PI|$ .

```

Fixem-nos que en aquest codi hem omès el cas en què $n \rightarrow \text{valor} > v$ perquè és simètric al cas $n \rightarrow \text{valor} < v$ (tinguem en compte que com que la pre especifica que $v \notin PI$, aleshores l'else ha de ser necessàriament $n \rightarrow \text{valor} > v$).

Per tant, farem la demostració només per la banda dreta, assumint que per a la banda esquerra serà simètrica.

Procedim a fer la verificació per passos:

1. Cas base. El cas base serà el bloc de codi de les línies 10 – 14. Tinguem en compte que en aquesta cas es compleix la pre, a més de les condicions $n \rightarrow \text{valor} < v$ i $n \rightarrow \text{fd} \neq \text{NULL}$. En aquest cas, el que fem és afegir un nou node amb l'element v , cosa que satisfà $v \in PI$ i $|A| + 1 = |PI|$. Com que és l'únic canvi, també es compleix $\text{subArbre}(A, PI)$, i també es compleix $\text{arrel}(fe) \leq x \wedge \text{arbreCerca}(fe)$ per la precondition. Finalment, com que $n \rightarrow \text{valor} < v$ i $n \rightarrow \text{fd} \neq \text{NULL}$, i també $x \leq \text{arrel}(fd)$ i $\text{arbreCerca}(fd)$ perquè només té un element.
2. Les crides recursives es fan correctament. Això consisteix en: (i) veure que els paràmetres es poden avaluar sempre. Com que $n \rightarrow \text{fd} \neq \text{NULL}$ llavors $n \rightarrow \text{fd}$ es pot avaluar.
(ii) verificar que es compleix amb la precondition de les crides recursives. Com que no hem fet encara cap canvi al PI , la precondition es compleix de manera trivial, ja que totes les condicions són sobre PI .
3. Després de les crides recursives, arribem a la postcondició. Cal veure que la postcondició de la crida recursiva (H.I) és **exactament igual** a la postcondició de la funció.
4. Funció de fita.

En aquest cas podem fer servir l'alçada o el nombre d'elements de l'arbre. Simplement cal veure que, si l'arbre no és buit, llavors $\text{alçada}(\text{PI}) = \text{alçada}(\text{PI} \rightarrow \text{fe}) + 1$. Aquesta equació està definida si A és un arbre $\neq \text{NULL}$, cosa que es verifica per la protecció de la condició de la crida recursiva.